Project95 Sensor driven navigation and visual docking

UTS 41014 Assessment 4 Track A Simulation project

Hanlin Wang Project95

This project combines sensing, state estimation and feedback control to move a MuJoCo TurtleBot3 Waffle Pi through a room with obstacles and stop at a target pose in front of the station. The robot builds a map from 2D LiDAR scans, estimates its pose from noisy wheel encoders and rendered ArUco images, and uses that estimate for navigation and docking. Parts A to E describe the model, sensors, EKF and controllers, followed by their test results.

All 24 first attempts in the final evaluation completed the task with zero recorded obstacle-contact steps. Final position errors were 6.052–12.041 mm, and the maximum absolute final yaw error was 1.495°, meeting the project acceptance criteria. Some runs nevertheless had large localization errors along the route, and the filter covariance was too small. The analysis therefore considers whole-run localization and final docking separately.

1 Task assumptions and acceptance

The robot first places local LiDAR scans at its estimated pose to build an occupancy grid. It then plans a route to an approach pose before switching to image-based full-pose docking. Encoder and estimator states are retained through the handoff. The room contains fixed geometry; one test condition also introduces a board at simulation time 2 s.

The station marker is at [2.85, 1.40, 0.25] m, and the docking target is [2.30 m, 1.40 m, 0 rad], 0.55 m in front of the station. The robot stops at this target and completes visual alignment. This distance is a project setting. Following problems with close-range camera views, the complete-task camera height was changed to 0.17 m and the resulting close-up rendered images were checked.

Task completion requires position error ≤25 mm and absolute yaw error ≤3°, with true translational speed ≤0.005 m/s and true yaw rate ≤0.02 rad/s over the checked stop tail. The task must also finish within 180 s, record zero obstacle-contact physics steps, and pass the clearance and actuator checks. These thresholds are defined for this project.

Wheel encoders → EKF prediction
Rendered RGB → ArUco corners → PnP → EKF correction
LiDAR and estimated pose → occupancy grid → A* route
Estimated pose and P → navigation or docking → wheel PI → actuators
Simulator truth → sensor generation and separate evaluation

Experiment scopes

Experiment	Content	Purpose
Early module runs	14 s EKF comparison; three 24 s trajectory-tracking modes.	Assess estimation or tracking performance, without a complete docking task.
No-path recovery after a board appears	Compare the original method, relocalization with map rebuilding, and added local subgoals; also examine saved Q/R replays and terminal-gain cases.	Diagnose failures and parameter effects. Development runs that reuse a seed are not counted as new success-rate trials.
Final varied-condition evaluation	Four conditions × two starts × three seeds; 24 first attempts.	Compare complete-task outcomes across conditions and starts, retaining every attempt.
Reproduction demonstrations	Normal room, appearing obstacle and docking-stage occlusion; repeated demonstration seed 4101901.	Check that the complete task runs after installation, separately from the 24-trial evaluation.

The module experiments examine estimation and trajectory tracking separately, while the complete task checks the transition from navigation to docking. Camera height and other conditions differ between the early modules and the complete task, so their results cannot be pooled. Configurations, data sources and metric definitions are recorded in evaluation/experiments.json. All code and data paths below are relative to the submission folder Project95.

Localization error, tracking error and final docking error answer different questions. Localization error compares the estimate with truth at the same time; tracking error compares the actual motion with the reference trajectory; final docking error compares only the end pose with the target. Position RMSE is the square root of the mean squared Euclidean position error. Yaw differences are wrapped to [-pi, pi) before calculating RMSE. These measures need to be considered together: an accurate final pose does not mean localization was accurate throughout the run.

2 A Modelling and structural analysis

The reduced state is $q = [x, y, \theta]^T$, with shape 3×1 and units m, m and rad. Inputs are body forward speed v (m/s) and yaw rate ω (rad/s), calculated from the left and right wheel angular rates. Nominal wheel radius r = 0.033 m and track width b = 0.288 m come from the fixed model geometry.

$$v = r(\omega L + \omega R)/2 \qquad \omega = r(\omega R - \omega L)/b$$
$$\dot{q} = [v\cos\theta, v\sin\theta, \omega]^T$$
$$\Delta s = r(\Delta\varphi L + \Delta\varphi R)/2 \qquad \Delta\theta = r(\Delta\varphi R - \Delta\varphi L)/b$$
$$q_{k+1} = [x + \Delta s\,\mathrm{sinc}(\Delta\theta/2)\cos(\theta+\Delta\theta/2),$$
$$\quad y + \Delta s\,\mathrm{sinc}(\Delta\theta/2)\sin(\theta+\Delta\theta/2), \mathrm{wrap}(\theta+\Delta\theta)]^T$$
$$\mathrm{sinc}(a) = \sin(a)/a, \text{ with } \mathrm{sinc}(0) = 1$$

A planar differential-drive model is used, assuming constant curvature within each sampling interval. It calculates pose changes directly from wheel increments for EKF prediction and feedback control. Slip, friction and motor response lag are not fully represented, so predicted motion differs from the physical simulation. Offline geometry and residual analysis was used to examine this difference. The online estimator retains the nominal wheel parameters rather than values fitted to simulator truth.

Wheel radius and track width come from the pinned ROBOTIS model [1]. Differential-drive modelling and structural analysis draw on [4]. The failure of feedforward wheel rates to produce accurate tracking in Section 5 also shows the difference between simplified kinematics and physical response.

Controllability

For a straight reference at $\theta = 0$ with nonzero forward reference speed v_o, the local linearization of the body-input model is:

$A = [[0,0,0], [0,0,v_o], [0,0,0]]$
$B = [[1,0], [0,0], [0,1]]$
$\text{rank}([B, AB, A^2B]) = 3$ when $v_o \neq 0$; rank = 2 at $v_o = 0$.

At rest, the linearized model cannot directly correct lateral error because the differential-drive robot cannot move sideways. It can still reach a lateral position by turning and moving forward, so rank 2 does not mean that position is unreachable. This analysis concerns the local kinematic model. Estimation error, mode switching and physical dynamics must also be considered using the simulation results below.

Observability

For a fixed landmark centre (X_i,Y_i), range is $\sqrt{((X_i-x)^2+(Y_i-y)^2)}$ and bearing is $\text{wrap}(\text{atan2}(Y_i-y,X_i-x)-\theta)$. A single centre leaves a rotational ambiguity about the landmark, giving rank 2; the corresponding time-varying analysis retains this ambiguity. Two distinct known centres, ArUco IDs 17 and 18 separated by 0.60 m, provide rank 3 for this route. The initial image and declared fixed map together supply the pose prior.

With a known marker orientation, PnP can also provide a planar pose measurement $z = q + \text{noise}$ with $H = I_3$. Recovery and docking use this measurement to add the orientation constraint unavailable from a single landmark centre. The result is still affected by PnP ambiguity, viewing distance and image bias.

Implementation: evaluation/A_Modelling/code/state_model.py.

3 B Sensor calibration assumptions and frames

Sensor configuration

Sensor or item	Value and basis	Limitations
Wheel encoders	100 Hz; synthetic increment variance = $0.001\|\Delta\phi\| + 0.000001\Delta t$ rad²; PCG64 seed recorded for each run.	Designed synthetic noise, not real-encoder calibration. Target wheel speed is not measured wheel speed.
2D LiDAR	10 Hz; 360 beams at 1° intervals; range 0.05–4 m; base mounting [−0.049,0,0.17] m.	Ideal instantaneous rays; no added range noise, scan-motion distortion or dropout. No-return and too-close states are distinguished.
RGB camera	10 Hz; 960×720; vertical field of view 60°; zero distortion; complete-task mounting [0.10,0,0.17] m.	Ideal intrinsics and known mounting checked in simulation, not real-camera calibration. The complete-task configuration overrides the earlier 0.12 m height in the base camera file.
ArUco observations	DICT_4X4_50; IDs 17/18; black-border side length 0.20 m. Actual rendered RGB → detected and refined corners → PnP.	Quantization, clipping, marker loss and image-geometry bias remain. Optical-axis depth is not horizontal range in the robot base frame.

Camera intrinsics are calculated from image resolution and field of view using the pinhole model. Static tests compare detected corners, range and bearing, and image-derived pose with known scene geometry. Occluded markers and markers behind the camera produce no observation. LiDAR ranging geometry is checked at four static poses. These tests examine the simulated sensor configuration and frame transforms; no physical calibration or hardware noise measurements were made. Detection and pose calculations use the OpenCV documentation [3].

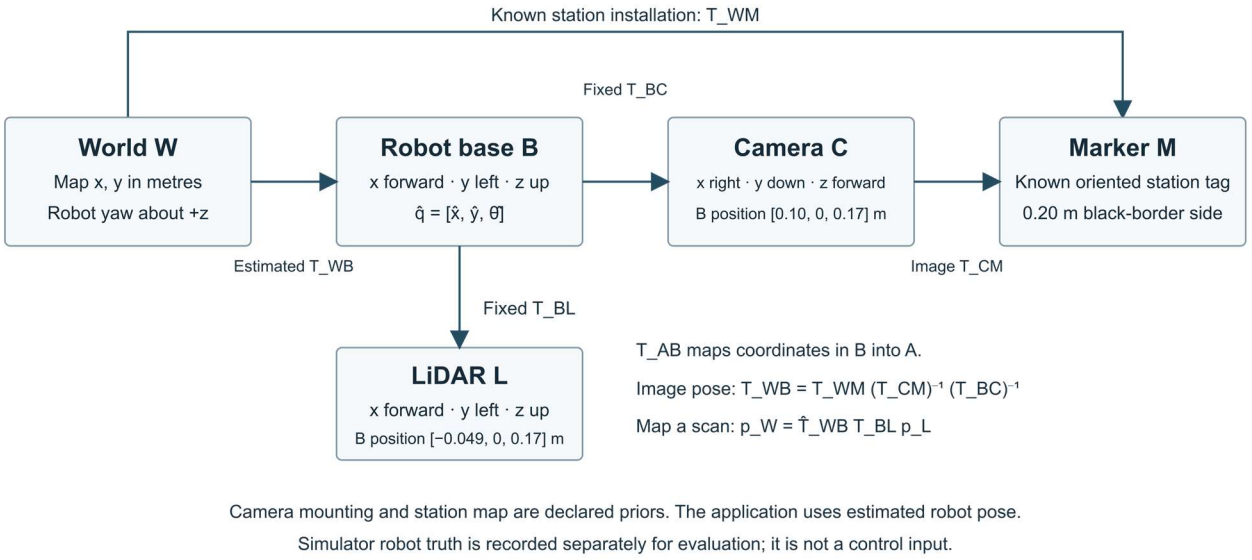


Figure 1 Declared sensor mounting and station transforms. Scan mapping uses the estimated base pose in the world frame; the application does not receive the simulator robot's true pose.

Image pose error across rendered views

The image tests use four station separations, each with nine combinations of lateral offset and yaw. Position error is the Euclidean distance between the image-derived planar position and the known test position. At world-x separations of 0.55, 0.85, 1.30 and 2.00 m, position RMSE is 0.587, 4.358, 16.257 and 43.838 mm, respectively. Error increases with distance, so close-range docking accuracy cannot represent visual accuracy along the whole route.

At greater distances, the marker occupies fewer pixels, so the same corner-location error produces a larger change in the inferred pose. The 36 fixed views are consistent with this relationship. They also show that constant observation noise misses the effects of view and distance. These tests do not repeatedly sample random noise; they measure error variation under fixed geometric conditions.

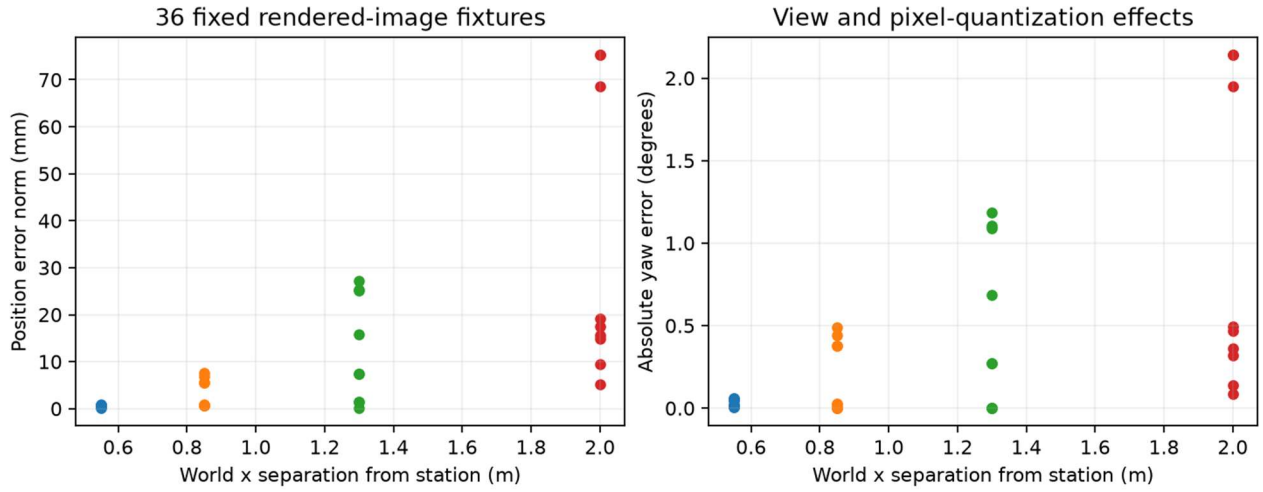


Figure 2 Image-derived pose error across 36 fixed rendered views. The horizontal axis is world-x separation from the station, not radial range. Data: evaluation/B_Sensing/data/camera.csv.

4 C Estimator design tuning and performance

Encoder increments predict q and its 3×3 design covariance P at 100 Hz, maintaining the estimate between camera observations. Motion Jacobians F and G propagate the state and wheel-increment noise, with an

added model-diffusion term for unmodelled motion. New camera observations are processed at 10 Hz using innovation/NIS checks and the Joseph covariance update:

$$P^{-} = FPF^{T} + G\Sigma\varphi G^{T} + \mathrm{diag}([0.003^2, 0.003^2, 0.015^2])\Delta t$$
$$K = P^{-}H^{T}(HP^{-}H^{T} + R)^{-1}$$
$$P^{+} = (I-KH)P^{-}(I-KH)^{T} + KRK^{T}$$

Q describes uncertainty in motion prediction, while R describes uncertainty in the visual measurement. They affect the size of visual corrections and whether observations pass the innovation gate. P is the state covariance calculated during prediction and update. Whether the noise model is suitable needs to be checked against actual error, rather than only whether P decreases. The EKF calculation follows [5].

Model-diffusion standard deviations are [0.003,0.003,0.015], with respective units m, m and rad per √s. Range/bearing R uses independent standard deviations [0.02 m,0.25°] and a same-frame shared component [0.01 m,0.15°], retaining off-diagonal correlations. Full-pose R is propagated from eight corner coordinates using σ = 0.5 pixel, 0.05-pixel central differences and pose standard-deviation floors [0.005 m,0.005 m,0.015 rad]. These are simulation design settings, not values calibrated from hardware data.

Wheel-increment noise follows the synthetic encoder model, while the extra model-diffusion term accounts for changes omitted by the kinematics. The same-frame shared term represents correlation between detections in one image. Corner propagation makes full-pose R depend on viewing geometry, and the standard-deviation floors prevent excessively small measurement covariance. These measures still do not fully describe cross-frame bias; the results below show actual errors exceeding the model's expectations.

Navigation normally uses range and bearing to one or two landmark centres. Recovery and docking use the full pose of an oriented marker, as distinguished in Section 2. Each RGB image is processed once, including images used for initialization or rejected by the gate. This avoids updating twice with range/bearing and full-pose information from the same image. Without a marker, the EKF predicts from encoders alone.

In the obstacle demonstration in Section 6, whole-run localization RMSE is 98.956 mm, while final docking error is only 11.690 mm. Later visual alignment can correct the end pose even though localization along the route remains problematic. In the no-path recovery case, the actual pose falls inside the nominal 95% ellipsoid for only 74.3% of time samples, indicating that the covariance underestimates error in this run. P and the ±2 standard-deviation bands therefore remain uncalibrated model uncertainty, not reliable confidence bounds.

Estimator module comparison and uncertainty

The 14 s estimator experiment compares the EKF with encoder-only odometry using the same initial state and measured wheel inputs. From 4 to 7 s, both markers are physically visible, but the program withholds one marker's messages to simulate partial message loss. From 7 to 10 s, a physical board occludes the markers and all raw detections disappear.

Without vision, the EKF continues encoder prediction and its uncertainty grows. Visual corrections resume when the markers become visible again. This module uses a camera height of 0.12 m, compared with 0.17 m in the complete task, so the table compares the EKF and odometry only under these module conditions.

Metric	Encoder only	EKF
Position RMSE mm	13.410	5.438
Yaw RMSE °	0.850	0.620
Final position error mm	16.127	3.586

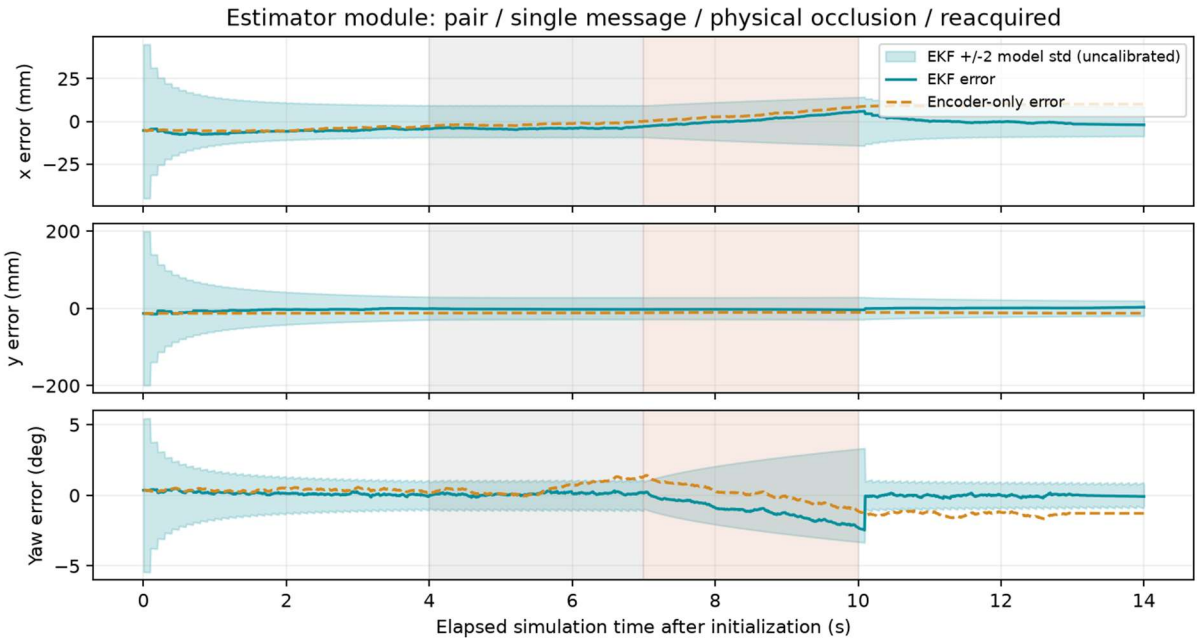


Figure 3 Errors relative to simulator truth and the EKF ±2 standard-deviation bands. Grey shading marks delivery of only one marker's messages; warm shading marks physical occlusion. The bands are calculated from P and remain uncalibrated. Data: evaluation/C_Estimation/data/ekf.csv.

Adding vision reduces position RMSE from 13.410 to 5.438 mm and final position error from 16.127 to 3.586 mm. Figure 3 shows error accumulating during encoder prediction when measurements are missing, then being corrected when vision returns. Overall position error is lower with the EKF in this experiment, but errors in individual intervals and directions do not always decrease together.

Effects of Q and R on the estimate

This comparison uses saved sensor inputs and changes either model-diffusion Q or visual R variance to one quarter or four times its original value. Only one term changes at a time; wheel noise and recovery times remain fixed. This isolates the effect of filter weighting for the same inputs. The robot does not move in response to the new estimates, so these are fixed-input replays rather than closed-loop task results with new weights.

In the no-path recovery case, model Q ×4 reduces position RMSE from 57.651 to 51.564 mm and NIS-rejected frames from 32 to zero, but yaw RMSE rises slightly from 6.481 to 6.494°. In another comparison case, visual R ×4 instead raises position RMSE from 3.322 to 5.389 mm. The same adjustment therefore affects cases and metrics differently. Fewer rejected frames do not establish more accurate covariance. These replays have not tested how changed weights would affect the closed-loop route or recovery timing.

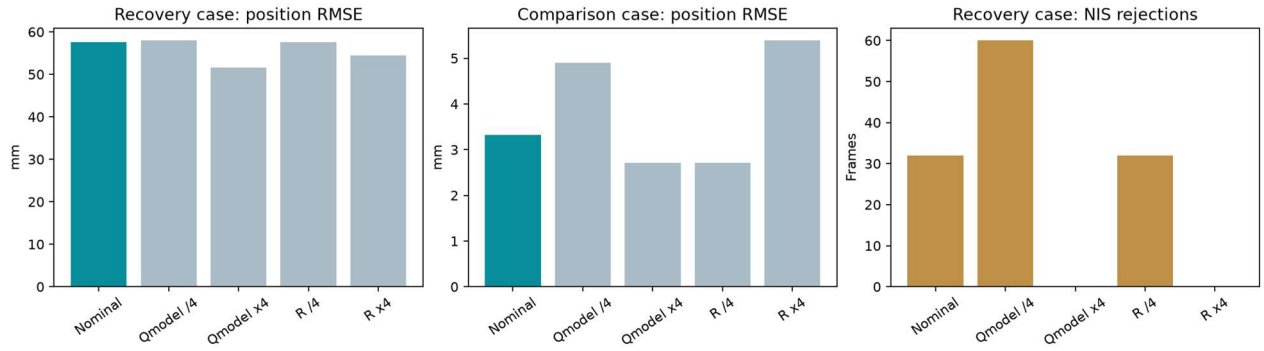


Figure 4 Q or R changes using the same saved inputs. The left and right plots show the no-path recovery case; the middle plot shows another comparison case. Model Q refers only to the extra model-diffusion term. Data: evaluation/C_Estimation/data/filter_replay.csv. Plotting uses saved replay summaries and does not rerun the estimator.

5 D Control using estimated state

The trajectory module expresses reference error in the estimated robot body frame as $e = [e_x, e_y, e\theta]$ and applies the following state-feedback law:

$$v = v_{ref}\cos(e\theta) + k_x e_x$$
$$\omega = \omega_{ref} + k_y v_{ref} e_y + k_\theta \sin(e\theta)$$
$$k_x = 0.9\ s^{-1},\ k_y = 120\ m^{-2},\ k_\theta = 1.4\ s^{-1}$$

The outer loop runs at 20 Hz and holds commands between updates. The 100 Hz wheel PI loop uses filtered noisy encoder rates, with kp = 0.6, ki = 5 s^{-1}, a filter time constant of 0.08 s and conditional anti-windup. Actuator targets are limited to ±7.88 rad/s. Servo and friction dynamics mean that the requested wheel rate is not necessarily the rate achieved by the robot.

Wheel PI corrects the difference between target and measured wheel rates, while pose feedback corrects position and yaw deviations from the reference trajectory. Rate filtering reduces noise but adds lag. The following three-mode comparison assesses these gains; no optimal-gain search was performed. Background on conditional integration and obstacle inflation is given in [6].

Trajectory module comparison 24 s reference Seed 4101701

Mode	Position tracking RMSE mm	Final position error mm	Yaw tracking RMSE °	Final yaw error °
Feedforward only feedforward_only	533.710	829.147	2.749	+2.291
Wheel PI only wheel_PI_only	65.285	77.656	2.813	+2.669
Pose feedback pose_feedback	19.200	4.136	2.914	+0.111

Adding wheel PI reduces position tracking RMSE from 533.710 to 65.285 mm. Adding pose feedback reduces it further to 19.200 mm. With feedforward alone, the robot barely moves: the wheel rates calculated from the kinematics differ substantially from the physical response. The two feedback levels improve position tracking, but adding pose feedback increases yaw RMSE slightly from 2.813 to 2.914°. These gains therefore improve position more clearly than yaw tracking.

The three modes share the same initial physical setup and seed, but their motions differ. Noise also depends on wheel increments, so their sensor inputs are not identical. These experiments assess trajectory tracking and do not include docking. Physical actuator saturation does not occur in these runs; limit handling is checked in separate boundary tests.

Trajectory comparison for the three control modes

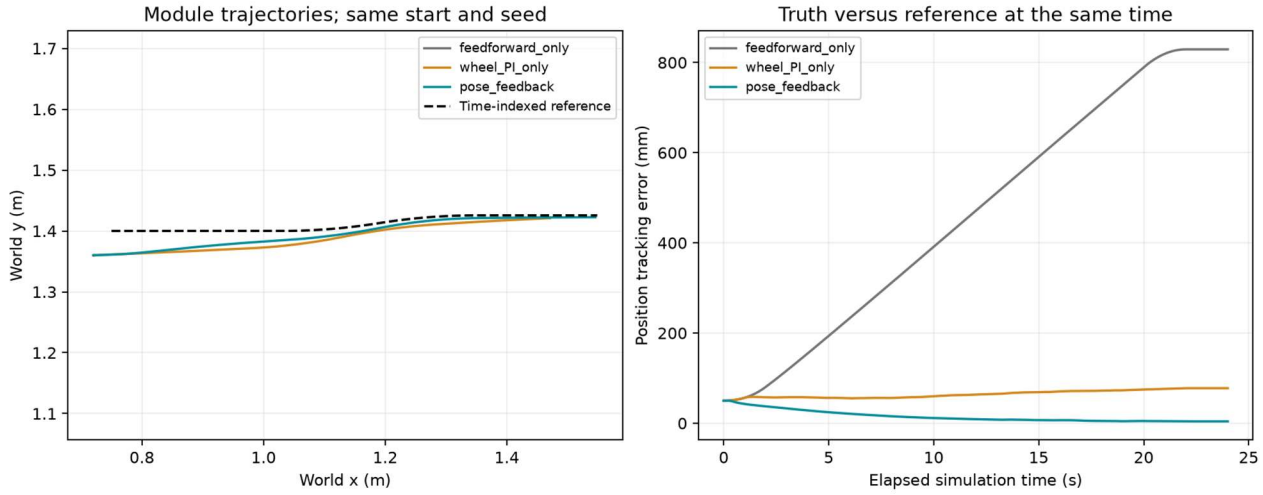


Figure 5 Three 24 s trajectory-module runs. Position error is relative to the reference at the same time, not the nearest point on the path. Data: evaluation/D_Control/data/tracking_*.csv. The initial setup and seed match, but motion and sensor-noise increments differ.

For the ideal model with fully known state, no saturation and no slip, $V = (e_x^2+e_y^2)/2 + (1-\cos e\theta)/k_y$ gives $\dot{V} = -k_x e_x^2 - (k\theta/k_y)\sin^2 e\theta \le 0$. Local persistent forward motion supports convergence reasoning, but this does not prove global stability of the system with estimation error, actuator lag, stopping or obstacles.

Complete-task route and terminal control

The complete task uses estimated-pose heading/distance feedback to follow A* waypoints on the scan-built occupancy grid. The respective gains are 1.4 s^{-1} and 0.8 s^{-1}, with cruise speed 0.045 m/s and yaw limit 0.22 rad/s. Terminal docking switches to a polar full-pose controller with $k\rho = 0.25\ s^{-1}$, $k\alpha = 0.8\ s^{-1}$ and the term $-k\beta\beta$, where $k\beta = 0.2\ s^{-1}$. Docking speed limits are 0.025 m/s and 0.18 rad/s. Both stages use the same encoder wheel PI loop.

Terminal gain trade-off

The terminal-gain comparison reuses the no-path recovery case, keeping $k\alpha$ and $k\beta$ fixed and changing only $k\rho$. Terminal duration starts at the switch to docking and excludes navigation. Doubling nominal $k\rho$ reduces duration from 33.85 to 28.15 s, but increases absolute final yaw error from 0.191 to 0.995°. Final position errors are similar across the three settings.

A larger approach gain finishes docking faster in this case but leaves greater yaw error. Distance to the target also increases briefly during the initial reorientation. Only one development case is compared, and the nominal setting shares the trajectory of the added-local-subgoal run in Section 6. These results are not enough to identify an optimal gain for other conditions.

kp s^{-1}	Terminal time s	Final position error mm	Final yaw error °
0.125	52.85	11.564	0.405
0.250	33.85	11.611	0.191
0.500	28.15	11.689	0.995

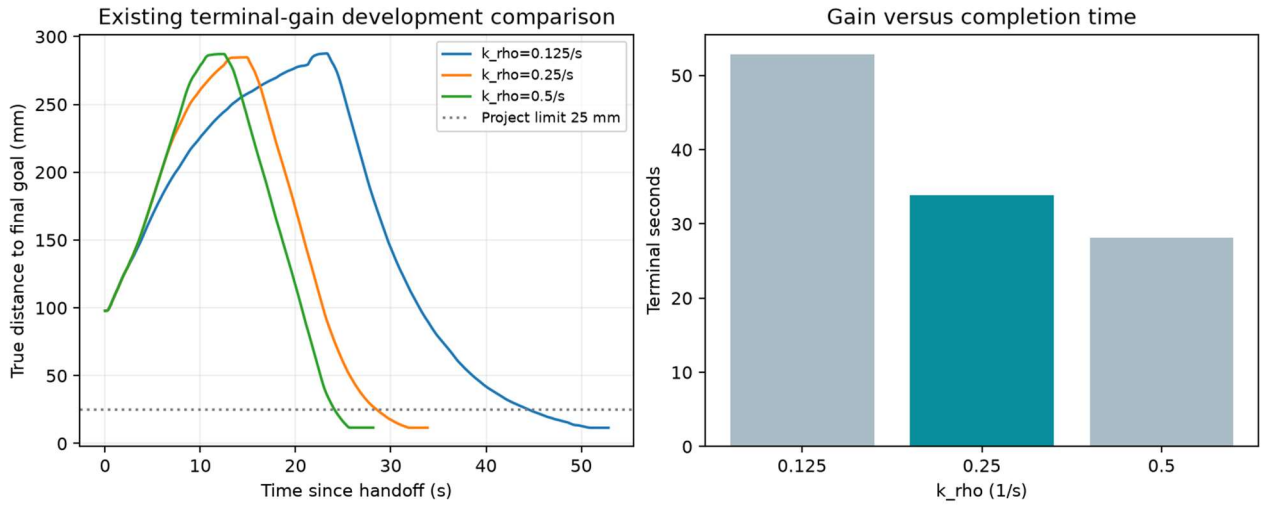


Figure 6 Target distance and duration in the terminal-gain comparison. The nominal setting uses the same development trajectory as the added-local-subgoal case in Section 6. Data under evaluation/: D_Control/data/gain_metrics.csv, gain_rho_*.csv and E_Integration_Robustness/data/recovery_v2.csv.

Mapping and no-path recovery

The occupancy grid uses 0.05 m cells and an inflation radius of 0.345355 m, combining a 0.25 m robot radius, a 0.06 m clearance margin and half a cell diagonal. Search and path simplification both block unknown cells and diagonal corner cutting. This conservative treatment reduces the space available for planning, which matters in the no-path recovery case. Scans or images older than 0.25 s, excessive design covariance or no known-free route produce a zero command. Docking requires the estimated pose to remain within 0.012 m / 0.025 rad for 2 s, followed by separate true-state acceptance under Section 1.

Persistent no-path recovery allows at most three attempts. The robot first stops, then checks that three consecutive full-pose image candidates agree. The three frames confirm relocalization; only the confirmation frame initializes pose and P, so P is not divided by three. Current measured wheel angles become the encoder baseline, mapping restarts, and the old route and arrival timing are cleared.

New scans rebuild the map at the corrected estimate. The start and candidate route still undergo the original footprint-inflation checks. If a complete route is unavailable, the robot first tries a local subgoal in observed free space to obtain further scans. It remains stopped if the start is still forbidden or no safe route exists. Recovery retains the original clearance margin and does not force forbidden cells to become free.

6 E Integration robustness and failure analysis

Complete-task demonstrations

Navigation and docking share continuous encoder, EKF and wheel PI states. LiDAR scans are placed at the estimated pose, and the planner finds waypoints from the resulting map. The map must be rebuilt after relocalization; otherwise, scans previously placed at an incorrect pose would continue to affect planning.

The following runs compare the complete task in a normal room, with an appearing board and with brief camera occlusion.

Complete tasks Repeated demonstration seed 4101901 Simulation time

Condition	Time s	Final position error mm	Final yaw error °	Localization RMSE mm	Contact steps
Normal room	48.80	11.620	-0.800	3.465	0
Appearing obstacle	99.75	11.690	+0.074	98.956	0
Docking camera occlusion	49.55	11.785	-0.505	3.421	0

The obstacle condition adds a board at simulation time 2 s. The occlusion condition physically blocks the camera during 0.5–1.5 s after docking begins. These three demonstrations reuse seed 4101901 to check the complete program and are not included in the 24-trial varied-condition evaluation below.

The obstacle task takes 99.75 s, considerably longer than the normal task's 48.80 s, and has greater localization RMSE. Both still finish about 11.6–11.7 mm from the target. Later visual docking achieves alignment but does not remove the problem of drift along the preceding route. The conditions and trajectories differ, so these results alone cannot isolate the contribution of each recovery step.

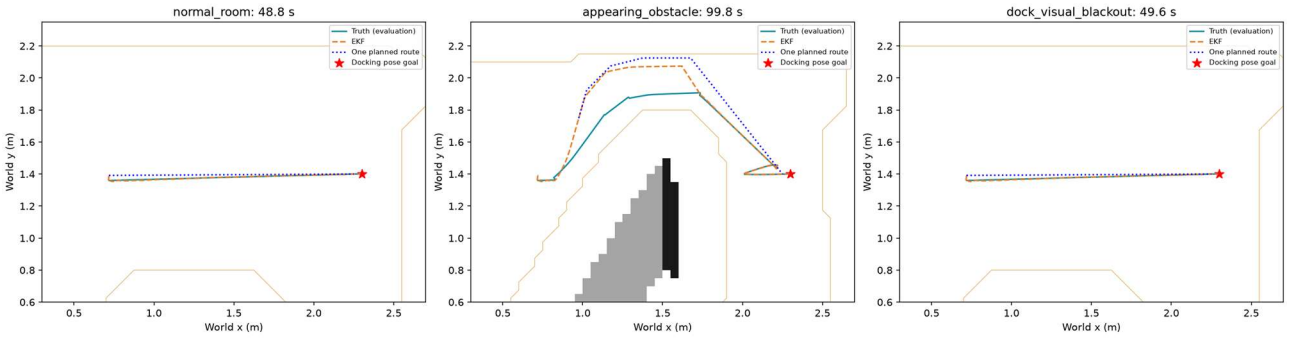


Figure 7 World coordinates in m. Solid teal shows evaluation truth, dashed orange the EKF estimate, dotted blue one planned route, and the red star the docking target. Localization error in the obstacle run remains visible even though final docking passes.

Results for the three demonstrations are saved in evaluation/E_Integration_Robustness/data/task_summary.json.

Final varied-condition evaluation

The final evaluation uses four conditions: normal operation, an appearing board, terminal occlusion and encoder variance ×4. Each condition uses two starts and three different seeds per combination, giving 24 pre-registered first attempts with seeds 4104001–4104024. All 24/24 attempts complete with zero recorded obstacle contacts. Final position errors are 6.052–12.041 mm, maximum absolute final yaw error is 1.495°, and task times are 48.70–113.30 s. Although all end poses pass acceptance, the largest whole-run localization RMSE is still 95.190 mm.

One run in the earlier 24-trial batch did not complete. After the board appeared, the robot could not find a feasible route and eventually timed out, giving 23/24 completions. This failure led to recovery changes using full-pose relocalization, map rebuilding and local subgoals, compared below. The final evaluation uses the modified program and a new set of seeds to assess it across conditions. Earlier development runs and installation demonstrations are not included in these 24 trials.

No-path recovery after a board appears

Methods compared on the same no-path recovery case Seed 4102008

Version	Outcome	Time s	Goal error mm	Localization RMSE mm
Original	Timeout	180.00	1011.390	119.895
Relocalization and map rebuilding	Timeout	180.00	1096.400	48.371
Added local subgoals	Completed	119.05	11.611	57.651

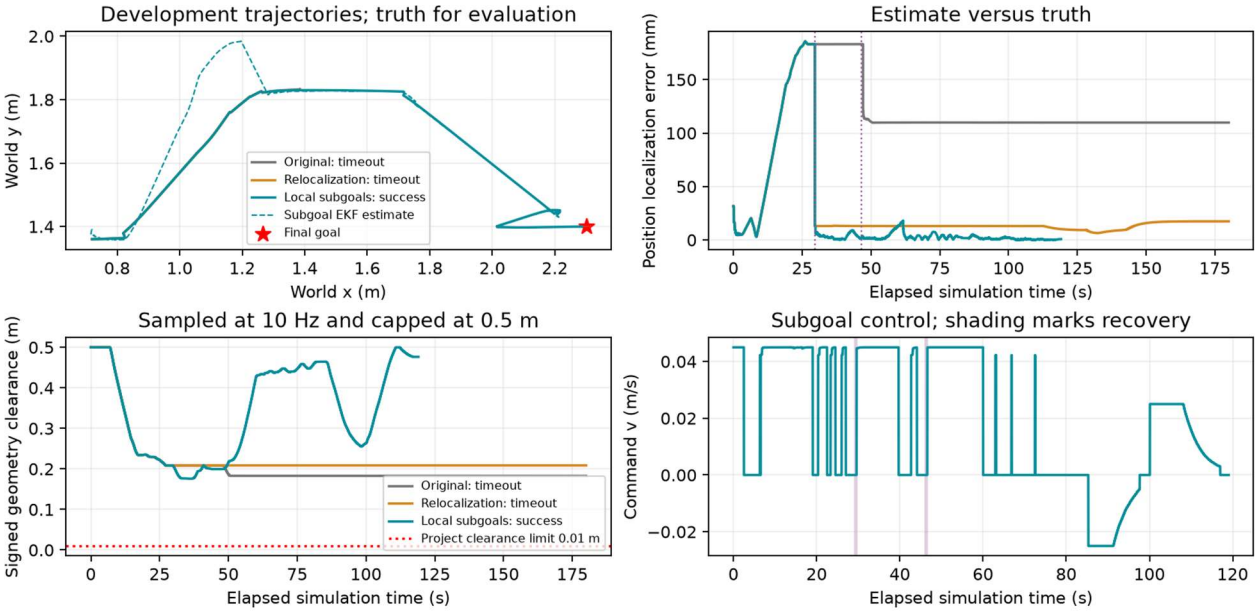


Figure 8 The original method and two recovery methods after the board appears. Shading marks stationary recovery and vertical lines mark pose confirmation. Obstacle clearance is sampled at LiDAR times and capped at 0.5 m in the plot; contacts are recorded separately at physics steps. Data: evaluation/E_Integration_Robustness/data/recovery_*.csv.

In the original method, the cell containing the estimated position is free before inflation, but becomes forbidden after accounting for the robot footprint and clearance margin. The same blocking occurs when rebuilding from the current scan alone, showing that removing old scans is insufficient. With only one landmark centre visible, small measurement innovations can also coexist with a large overall pose error, consistent with the rotational ambiguity in Section 2.

The first change confirms full pose using an oriented marker and rebuilds the map. Localization RMSE decreases, but the known-free region still does not connect to a route to the target, so the task times out again. The second change allows the robot to visit a local subgoal in known-free space, acquire new scans and then replan to the target. This run completes in 119.05 s with final position error 11.611 mm and yaw error 0.191°.

Relocalization without local subgoals gives localization RMSE of 48.371 mm, lower than the 57.651 mm with local subgoals, yet does not reach the target. A smaller average localization error does not mean the planning problem is solved. Different trajectories and stopping periods also affect that average, so it needs to be read alongside the time histories in Figure 8. The three methods reuse the same random seed. This comparison diagnoses the failure case and does not add independent success-rate trials.

Limits of the varied-condition evaluation

The added persistent no-path recovery branch is never triggered in the final 24 trials. Each of the six obstacle runs triggers the existing two-point recovery once. The 24/24 result therefore does not directly test the new branch or establish its performance in other situations. Each condition/start combination has only three runs, and zero contacts applies only to these tested trajectories.

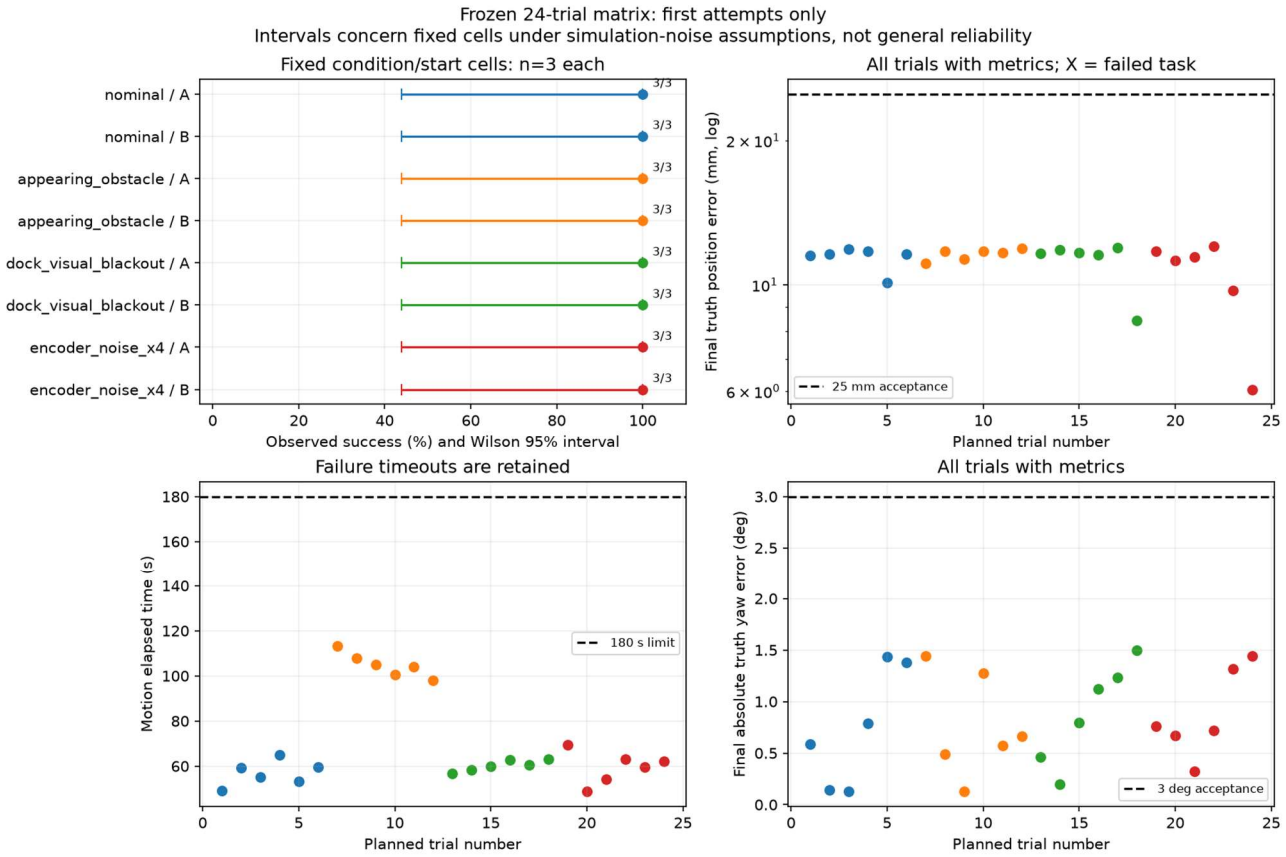


Figure 9 All first attempts in the final varied-condition evaluation. The 24/24 outcome applies to this finite set of conditions, with only three runs per condition/start combination.

Metrics for all 24 trials: evaluation/E_Integration_Robustness/data/trial_metrics.csv. Varied-condition batch summary: evaluation/E_Integration_Robustness/data/robustness_summary.json.

7 AI declaration and references

Generative AI declaration

I used OpenAI Codex for step-by-step explanations in Chinese to help me understand the course requirements, relevant concepts, and the mathematics and programming involved in the project. I also used it to assist with Chinese–English translation of course materials and project documents. Codex assisted with designing, writing and debugging Python programs for the differential-drive model, encoder/vision EKF, trajectory feedback, wheel PI, navigation, visual docking, evaluation metrics and parameter comparisons. It was also used to check mathematical derivations, matrix dimensions and error conventions; execute tests and simulations; diagnose failures and explain numbers and figures; and check whether installation, execution and result analysis could be repeated.

I also used Codex to draft and revise the report and README, organize Chinese learning notes, and prepare the video storyboard.

References

- [1] ROBOTIS TurtleBot3 MuJoCo
- [2] Course UR3 tutorial
- [3] MuJoCo Python documentation; OpenCV ArUco; OpenCV PnP; NumPy; Matplotlib; Pillow
- [4] RoboticsBook Differential drive; Modern Robotics Controllability
- [5] Kleeman Kalman filter notes; MathWorks Extended Kalman filter documentation
- [6] Modern Robotics Feedback control; control_toolbox Conditional integration; Nav2 Inflation layer; Nav2 Obstacle layer; HMC Polar feedback example